**Escuela Politécnica Nacional**




**Tema**

Ejercicio Tres “Contador De Líneas Y Caracteres”

**Asignatura**

Compiladores y Lenguajes

**Integrantes**

Marco Mendieta
Vanessa Paucar
Sebastián Ríos
André Rosero

**Docente**

Ing. Rodrigo Chancusig MSc.

**Paralelo**

GR1CD

**Fecha**

07/11/2026

**Archivo fuente**

tres.l

## Objetivo

Implementar y ejecutar un analizador léxico para el conteo de líneas y caracteres por medio de especificaciones de Flex, lex y gcc, de esta forma verificando el conteo correcto de saltos de línea y caracteres, además comparar su estructura y eficiencia en cuanto a la sintaxis del compilador frente a un código equivalente y generado mediante inteligencia Artificial.

## Desarrollo de la actividad

### 1. Preparación del entorno

Para desarrollar del presente ejercicio en un sistema operativo en base a Linux, se creó un entorno de trabajo dentro de la carpeta principal *IB/* la cual contiene el directorio *TRES*, y en el cual se almacenarán todos los archivos que se crearon para la ejecución del ejercicio.

Desde la terminal y por medio de comandos principales en Linux como: mkdir, cd y nano, se creó y direcciono el espacio de trabajo a el directorio TRES.

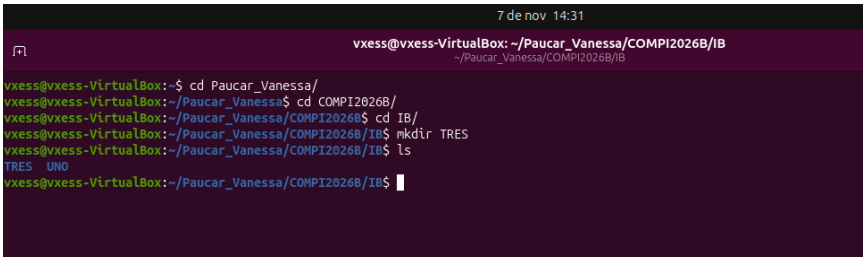



***Figura 1:*** *Navegación por directorios hasta la carpeta IB/ y creación de directorio TRES.*

Finalmente, el esquema completo que va a contener nuestro directorio TRES es el siguiente:

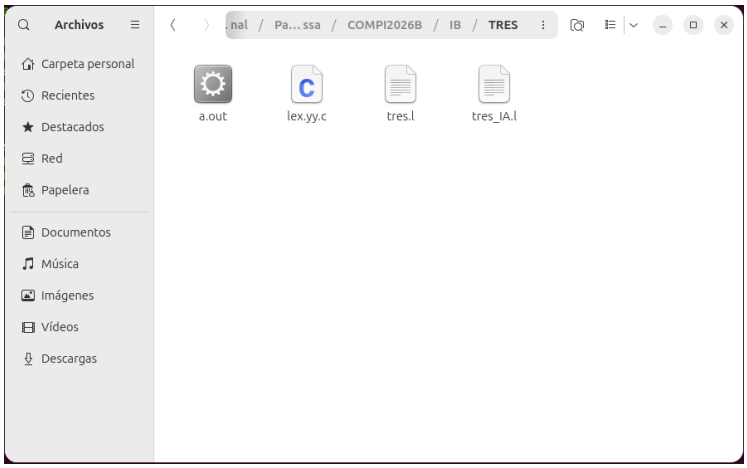



***Figura 2:*** *Archivos almacenados dentro del directorio TRES.*

- **lexx.yy.c y a-out:** archivos que fueron generados automáticamente por el compilador al momento de la ejecución.
- **Tres.l:** Programa fuente para el conteo de líneas y caracteres.
- **Tres_ia.l:** Programa fuente generado con inteligencia Artificial para el conteo de líneas y caracteres.

### 2. Código fuente

El programa se divide en tres secciones importantes para el adecuado funcionamiento del *"Contador de líneas y caracteres"*, las cuales son:

1. *Sección de Definiciones*

Es la parte principal del código que está contenida dentro de la sintaxis: ***%{ … %}***, en la cual se declaran las librerías de C con la sintaxis *#include, para esta práctica se usa tres librerías:*

1. ***<stdio.h>:*** para el manejo de entradas/salidas.
2. ***<stdlib.h>:*** utilidades generales.
3. ***<string.h>:*** manipulación de cadenas

Además, la declaración e inicialización de las variables globales, *numlin,* para el conteo de líneas, y *numcar*, para el conteo de caracteres.

2. *Sección de Reglas*
   Es la parte fundamental del programa, dado que contiene el núcleo donde se define las expresiones regulares y lo que debe hacer el programa al encontrarlas.
   1. ***[\n]:*** Si encuentra un salto en línea incrementa el contador *numlin*.
   2. ***. :*** Si detecta cualquier carácter que no sea salto de línea incrementa *numcar*.
3. *Sección de Subrutinas de Usuario*
   Esta sección se encuentra dentro de *int main(){…},* una vez el programa empiece con la ejecución, va a ingresar a la función *yylex()* la cual comienza a leer la entrada de texto y a ejecutar simultáneamente la sección de reglas.
   De esta forma, cuando el texto llegue a su fin, el usuario deberá de ingresar el comando Ctrl + D para culminar el análisis del texto, de esta forma se imprimirán los resultados con el número de líneas y el número de caracteres.




***Figura 3:*** *Programa fuente tres.l "contador de líneas y caracteres".*

## 3. Compilación y ejecución

Una vez guardado el archivo *tres.l*, se procede a realizar la compilación del código:

1. *Generar el archivo .c con lex:*
   Mediante el comando *lex tres.l* se genera un archivo lex.yy.c el cual procesa las reglas léxicas y lo procesa, en si traduce ese archivo en un lenguaje estándar de programación para que sea entendido por un compilador de C.
2. *Compilar el archivo C:*
   Mediante el comando *gcc lex.yy.c -lft*, el compilador de C transforma el archivo *lex.yy.c* y lo enlaza a la biblioteca de Flex para que el analizador léxico funcione de manera correcta sin errores.
3. *Ejecutar el programa:*
   Mediante el comando *./a.out* se ejecuta el programa compilado tres y el sistema se queda esperando a que se ingrese cualquier texto para analizar, una vez culminado se selecciona Ctrl + D y el programa imprimí los resultados.

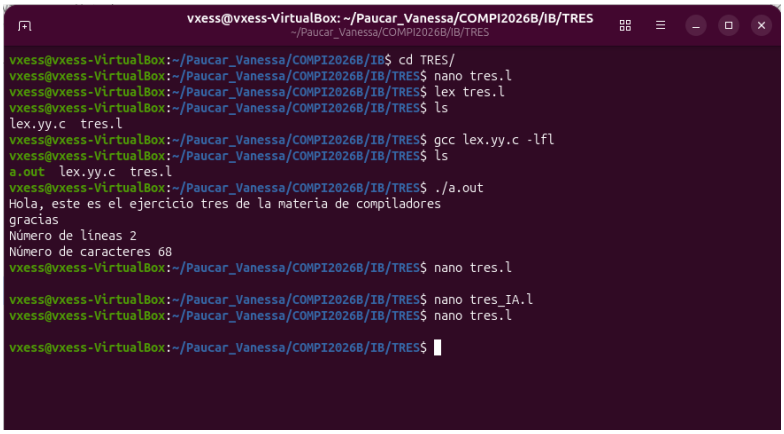



***Figura 4:*** *Comandos de compilación y ejecución para el programa* ***tres.l****.*

## 4. Pruebas y resultados

En este apartado compararemos netamente el programa dado por el material del docente y el programa dado por la IA:

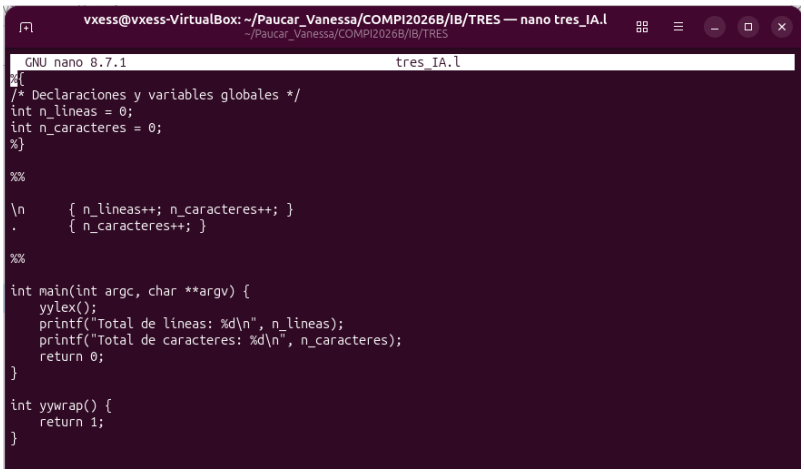



***Figura 5:*** *Código generado por la IA para el conteo de líneas y caracteres.*

Comparación de salida del programa con el mismo texto tanto en el programa *tres_IA.l* y *tres.l*

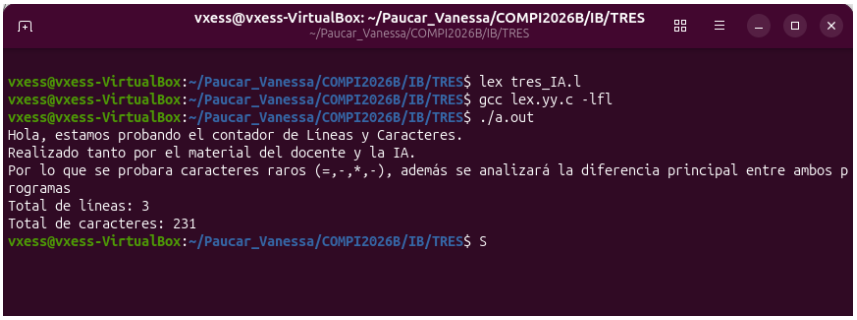



***Figura 6:*** *Comandos de compilación y ejecución para el programa* ***tres_IA.l****.*

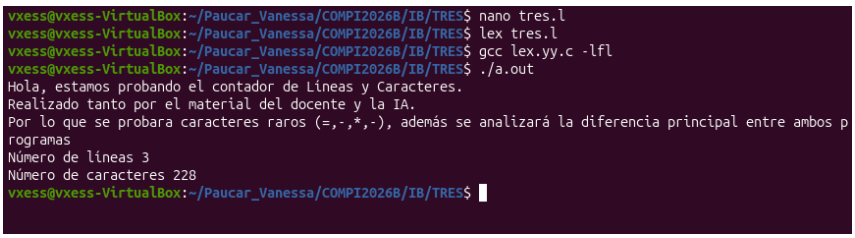



***Figura 7:*** *Comandos de compilación y ejecución para el programa* ***tres.l****.*

***Análisis de resultados***

A nivel de funcionamiento ambos programas funcionan de manera adecuada y similar, esperando que el usuario ingrese el texto mientras el *lex* realiza el análisis de los caracteres y las líneas, su principal diferencia radica en:

- *Sintaxis del código:* el programa *tres.l* incluye la declaración de las librerías a usar, mientras que el programa *tres_IA* opta por un enfoque implícito del entorno.
- *Semántica de conteo:* El programa tres_IA.l unifica el contador tanto de n_lineas y n_caracteres, por lo que de esta forma cada vez que exista un salto de línea, este va a modificar el contador a ambas variables, por ejemplo, este caso se evidencia tanto en la *figura 6* y la *figura 7*, los resultados varían en la cuenta de caracteres y es por este motivo.
- El programa *tres.l* emplea un main() simple sin definir yywrap(), lo que indica que delega por completo esa función a el flag, mientras que el archivo tres_IA.l incluye la firma completa con parámetros en el main y define explícitamente la función yywrap() { return 1; }

**Conclusiones**

Finalmente, se implementó y ejecutó un analizador léxico con Lex y GCC para contar líneas y caracteres en un entorno Linux. Se verificó que el programa realiza el conteo correcto mediante el uso de patrones regulares. Al comparar el código académico con uno generado por inteligencia artificial, se observó que ambos funcionan correctamente y cumplen el objetivo. La diferencia principal está en su estructura: el código impartido es más directo y posee las declaraciones necesarias para la ejecución correcta del programa, mientras que el de la inteligencia artificial incluye bloques de acción unificados y funciones auxiliares como yywrap. A pesar de estas diferencias, ambas versiones procesan la información con una eficiencia similar y logran contar correctamente los saltos de línea y los caracteres.

**Recomendaciones.**

Se recomienda probar el analizador léxico con casos especiales, como archivos vacíos, archivos con una sola línea sin salto final o entradas con caracteres especiales. Esto permite comprobar que los contadores funcionan correctamente en diferentes situaciones.

También es importante definir un mismo criterio para contar los saltos de línea, especialmente al comparar el código del docente con el generado por inteligencia artificial, para obtener resultados iguales.

**Referencias**


1. *GCC, the GNU compiler collection*. (s/f). Gnu.org. Recuperado el 7 de noviembre de 2026, de https://gcc.gnu.org/
2. *Manual de LEX*. (s/f). Ugr.es. Recuperado el 7 de noviembre de 2026, de https://ccia.ugr.es/~rosa/tutormc/lex/Lex.html